

# General

## Shell

- Install and configure **zsh**: [Link](#)

- Create a new file:

```
touch <filename>
```

- Create a range of files:

```
touch myfiles{1..10}.txt
```

- **grep** for filtering:

```
ls | grep <wanted-folder>
```

Use **-v** to exclude files.

- Check current user:

```
whoami
```

- Check hostname:

```
hostname
```

- Check hostname IP in LAN:

```
hostname -I
```

- Check current port usage:

```
sudo lsof -i -P -n | grep LISTEN
```

- Add alias permanently:

```
echo "alias aliasname='aliascommand'" >>  
~/.bashrc && source ~/.bashrc  
echo "alias aliasname='aliascommand'" >>  
~/.zshrc && source ~/.zshrc
```

- Add app to PATH:

```
echo 'export PATH=/your/directory/path:  
$PATH' >> ~/.bashrc && source ~/.bashrc
```

- Check disk usage:

```
df -h <your-path>  
sudo du -h --max-depth=1
```

- Check partitions:

```
lsblk
```

- Find in file contents:

```
grep -rnw yourpath -e "yourregex"
```

- Run **.jar** file:

```
java -jar yourjarfile.jar
```

- Network manager UI:

```
nmtui
```

- Check used ports:

```
netstat -tunlp
```

- Check proxies:

```
env | grep -i proxy
```

- Check port status:

```
nmap -p 5432 172.19.0.3
```

- Remove files recursively:

```
find . -name "FILENAMEWILDCARD" -delete
```

- Increase login timeout:

```
sudo visudo  
Defaults timestamp_timeout=240
```

- Run command in background:

```
yourcommand >output.log 2>&1 &
```

- Check background processes:

```
jobs
```

- Bring job to foreground:

```
fg %1
```

- Resume job in background:

```
bg %1
```

- Kill job:

```
kill %1
```

- Beautify JSON:

```
jsonoutputer | jq .
```

- View logs in real-time:

```
tail -f output.log
```

- Find and replace in file:

```
sed 's/pattern/replacement/g' file  
sed -i 's/pattern/replacement/g' file  
sed -i.bak 's/pattern/replacement/g' file
```

## systemd

- Start service:

```
systemctl start yourservice.service
```

- Stop service:

```
systemctl stop yourservice.service
```

- Restart service:

```
systemctl restart yourservice.service
```

- Enable service:

```
systemctl enable yourservice.service
```

- Disable service:

```
systemctl disable yourservice.service
```

## SSH

- Copy file from remote:

```
scp remoteuser@remotehostip:/path/to/file  
localdirectory
```

- Copy folder from remote:

```
scp -r remoteuser@remotehostip:/path/to/  
directory localdirectory
```

- Send file to remote:

```
scp filepath remoteuser@remotehostip:/  
remote/to/directory
```

- Send folder to remote:

```
scp -r folderpath remoteuser@remotehostip:/  
remote/to/directory
```

- Copy files with progress:

```
rsync -ah --progress source destination
```

## Security

### GPG (GNUPG)

- Generate keys:

```
gpg --gen-key
```

- List keys:

```
gpg --list-secret-keys
```

- Export keys:

```
gpg --export-secret-keys [key-id | email] >  
private_key.asc
```

- Import keys:

```
gpg --import private_key.asc
```

- Delete key:

```
gpg --delete-key [KeyID]  
gpg --delete-secret-key [KeyID]
```

- Symmetric encryption:

```
gpg -c --no-symkey-cache <filename>
```

- Asymmetric encryption:

```
gpg -e -r <key-email> <filename>
```

- Decrypt file:

```
gpg -d <filename>.gpg > <filename>
```

## Docker

### Installation

- TODO

### Docker Without Sudo

- Add user to docker group:

```
sudo groupadd docker  
sudo gpasswd -a $USER docker  
newgrp docker
```

## CentOS

- Remove old Docker versions:

```
sudo yum remove docker docker-engine docker  
.io containerd runc
```

- Install Docker:

```
sudo yum install -y yum-utils  
sudo yum-config-manager --add-repo https://  
download.docker.com/linux/centos/docker  
-ce.repo  
sudo yum install docker-ce docker-ce-cli  
containerd.io
```

- Manage Docker service:

```
systemctl start docker  
systemctl status docker  
systemctl stop docker
```

## CLI

- List Docker images:

```
docker images
```

- Remove an image:

```
docker rmi <image-name>
```

- Run a container:

```
docker run --name <container-name> -d <  
image-name>
```

- Stop a container forcefully:

```
docker kill <container-id>
```

- List running containers:

```
docker ps
```

- List all containers (running and stopped):

```
docker ps -a
```

- Remove stopped containers:

```
docker container prune
```

- Remove all containers forcefully:

```
docker rm -vf $(docker ps -aq)
```

- Remove all images:

```
docker rmi -f $(docker images -aq)
```

- Remove Docker cache and dangling data:

```
docker system prune
```

- Save an image to a file:

```
docker save -o <save-location>/<file-name>.  
tar <image-name>
```

- Load an image from a file:

```
docker load -i <image-file-path>.tar
```

## Volumes

- Default volumes path: `/var/lib/docker/volumes`
- Specify volumes in `docker-compose`:

```
volumes:
  <volumename1>:
  <volumename2>:
```

- Use `:rw` mode to avoid permission issues.

## Network

### Bridge

- Containers in the same bridge network can communicate using their names.

### Access Host from Container

- Add to `docker-compose.yml`:

```
extra_hosts:
  - "host.docker.internal:host-gateway"
```

- Or use flag:

```
docker run --add-host host.docker.internal:
  host-gateway
```

- Access host:

```
ping host.docker.internal
```

## Dockerfile

- Comment:

```
# this is a comment
```

- Copy files:

```
COPY <host-path> <container-path>
```

- Base image:

```
FROM <image-name>:<image-version>
```

- Set working directory:

```
WORKDIR <path-in-container>
```

- Environment variable:

```
ENV <var-name>=<var-value>
```

- Run shell command:

```
RUN <shell-command>
```

- Run command on container start:

```
CMD [ '<command>', '<arg1>', '<arg2>' ]
```

- Entrypoint:

```
ENTRYPOINT [ '<command>', '<arg1>', '<arg2>' ]
```

## docker-compose.yml

- Start services:

```
docker compose up
```

- Stop and remove services:

```
docker compose down
```

- Start in detached mode:

```
docker compose up -d
```

- Example `docker-compose.yml`:

```
# this is a comment
version: <version>

networks:
  <network-name>:
    driver: <network-type>

volumes:
  <volume-name>:

services:
  <service-name>:
    image: <image-address>
    depends_on: <dependencies>
    build: <dockerfile-path>
    deploy: <deploy-config>
    ports: <port-mapping>
    restart: <restart-policy>
    networks: <networks>
    environment: <environment-variables>
    volumes: <volumes-mapping>
    container_name: <container-name>
```

## Registry

- Push image to registry:

```
sudo docker tag <image-name> <your-registry>
  -address>/<image-name>
sudo docker push <your-registry-address>/<
  image-name>
```

## Docker Swarm

- Initialize Swarm:

```
docker swarm init
```

- Join Swarm:

```
docker swarm join --token <token> <manager-
  ip>:2377
```

- Deploy stack:

```
docker stack deploy -c docker-compose.yml
  my-stack
```

- Check stack status:

```
docker stack services my-stack
```

- Remove stack:

```
docker stack rm stack_name
```

- Remove service:

```
docker service rm service_name
```

- View logs:

```
docker service logs service_name
```

- Check service processes:

```
docker service ps service_name
```

- Check Swarm status:

```
docker info
```

- List all services:

```
docker service ls
```